○ E-LEARNING · STUDY NOTES

Nicloud · Kubernetes on AWS

Module 2 – Scaling, storage and inbound traffic

Contents

Learning objectives 2
Architecture 2
Karpenter: nodes on demand 2
Example: the Karpenter controller in this repo 3
NodePool and EC2NodeClass 3
Storage: StorageClasses 5
AWS Load Balancer Controller 5
Ingress and Gateway API 6
The five ingress controllers 6
ExternalDNS: automatic records 8
cert-manager: automatic certificates 9
Summary 10
Quiz 10
Additional resources 11

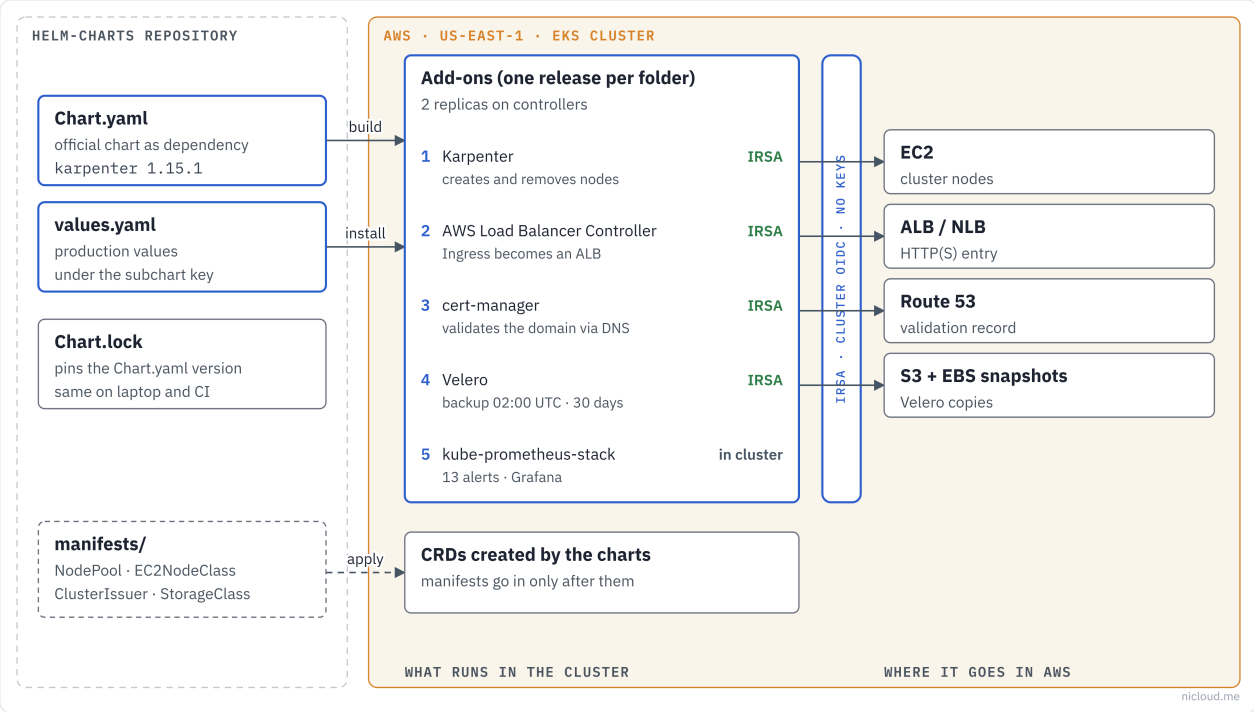
Learning objectives

In this module, you will learn to:

- Explain how **Karpenter** picks, creates and removes nodes from a **NodePool** and an **EC2NodeClass**.
- Configure gp3 and EFS **StorageClasses** and choose between `ReadWriteOnce` and `ReadWriteMany`.
- Describe what the **AWS Load Balancer Controller** creates in AWS from an Ingress or a Service.
- Compare ingress-nginx, Envoy Gateway, Traefik, Kong and ALB and pick one for each case.
- Tell the **Ingress** API from the **Gateway API**.
- Publish Route 53 records automatically with **ExternalDNS** without deleting what it does not own.
- Issue TLS certificates with **cert-manager** using the **DNS-01** challenge on Route 53.

Architecture

The diagram shows the path of a request: from the Internet to the load balancer, from the load balancer to the ingress controller, and from it to the pods. This module covers everything on that path, plus the nodes and disks the pods run on.



Karpenter: nodes on demand

Karpenter is the node autoscaler for EKS. It watches pods that are **Pending** for lack of room, works out what they ask for (CPU, memory, architecture, zone) and creates the EC2 instance that fits best, in seconds.

The difference from the classic Cluster Autoscaler is that Karpenter does not depend on node groups with a fixed instance type. It picks the type on the spot, from dozens of families, and it also works in reverse: it packs pods onto fewer nodes and shuts down what is left over. This is called **consolidation**.

The chart installs only the **controller**. What it is allowed to create is defined by two resources you apply afterwards: the **NodePool** (which nodes) and the **EC2NodeClass** (how to launch them in AWS).

Example: the Karpenter controller in this repo

```
# karpenter/values.yaml
karpenter:
  serviceAccount:
    annotations:
      eks.amazonaws.com/role-arn:
"arn:aws:iam::ACCOUNT_ID:role/KARPENTER_ROLE_NAME"
  replicas: 2
  settings:
    clusterName: "CLUSTER_NAME"
    clusterEndpoint: "CLUSTER_ENDPOINT"
  logLevel: info
```

- `replicas: 2`: one controller is active and the other is on standby (leader election). If the first one's node fails, the second takes over.
- The IRSA role must create and terminate instances, pass the node role (`iam:PassRole`) and read the Spot interruption queue.
- The two controllers must not run on nodes Karpenter itself created. If Karpenter removes the node it runs on, nobody creates the next one. Run them on a small managed node group or on Fargate.

NodePool and EC2NodeClass

The **NodePool** says **which nodes** may be created: capacity type (Spot or on-demand), instance families, architecture, limits and consolidation rules.

```
# karpenter/manifests/default-nodepool.yaml (excerpt)
spec:
  template:
    spec:
      requirements:
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["on-demand", "spot"]
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: ["c", "m", "r"]
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
  limits:
    cpu: "100"
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 30s
```

- With both Spot and on-demand allowed, Karpenter prefers **Spot** when capacity exists, because it is cheaper, and falls back to on-demand when it does not.
- `limits.cpu: "100"` is the NodePool ceiling: the vCPUs of all its nodes add up to 100 at most. It is the guard against a broken deploy asking for a thousand replicas.
- `WhenEmptyOrUnderutilized` with `consolidateAfter: 30s` removes empty nodes and replaces underused ones with smaller ones, 30 seconds after the situation appears.

The **EC2NodeClass** says **how to launch** the node: image, subnets, security groups and role.

```
# karpenter/manifests/ec2nodeclass.yaml (excerpt)
spec:
  amiSelectorTerms:
    - alias: al2023@latest
  subnetSelectorTerms:
    - tags:
        karpenter.sh/discovery: "CLUSTER_NAME"
  role: "KarpenterNodeRole-CLUSTER_NAME"
```

Karpenter discovers subnets and security groups by the `karpenter.sh/discovery` **tag**. Forgetting to tag the private subnets is the most common first-install mistake: the NodePool exists, but no node is ever born.

When not to use it

`al2023@latest` always takes the newest AMI, which can change the node image without notice on every consolidation. In more conservative production, pin a version (`al2023@v20261101`) and upgrade on purpose.

Karpenter's CRDs ship in the chart's `crds/` folder, and Helm does not upgrade them on `helm upgrade` (module 1). For a major upgrade, apply the new CRDs first, or install the separate `karpenter-crd` chart, which exists for this.

Storage: StorageClasses

A **StorageClass** tells Kubernetes how to create a disk when a pod asks for a volume (a **PersistentVolumeClaim**). The `storage/` folder has no chart, only two StorageClasses.

```
# storage/manifests/gp3-storageclass.yaml
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: gp3
  encrypted: "true"
allowVolumeExpansion: true
```

- **gp3** is the default EBS disk: cheaper than gp2, with performance that does not depend on size.
- `WaitForFirstConsumer` waits for the pod to be scheduled before creating the disk. An EBS volume lives in one zone only. If the disk were created first, it could land in a zone where the pod does not run.
- `encrypted: "true"` encrypts with the default EBS key.
- `allowVolumeExpansion` lets you grow the PVC without recreating the disk.

`efs` (EFS) covers the case EBS cannot: several pods, in different zones, reading and writing the same volume (`ReadWriteMany`). With `provisioningMode: efs-ap`, each PVC becomes an **access point** on an EFS file system you create beforehand (`fileSystemId`).

	gp3 (EBS)	**efs (EFS)**
Access	one node at a time (`ReadWriteOnce`)	many nodes (`ReadWriteMany`)
Zone	one	every zone in the region
Typical use	databases, Prometheus, Vault	shared files, uploads
Driver	EBS CSI (EKS addon)	EFS CSI (installed separately)

AWS Load Balancer Controller

The **AWS Load Balancer Controller** (`ingress-alb/` folder) watches the cluster and creates load balancers in AWS:

- an **ALB** for each Ingress with the `alb` class (or one ALB shared by several, with `group.name`);
- an **NLB** for each `LoadBalancer` Service with the `aws-load-balancer-type: external` annotation.

```
# ingress-alb/values.yaml
aws-load-balancer-controller:
  clusterName: "CLUSTER_NAME"
  serviceAccount:
    annotations:
      eks.amazonaws.com/role-arn:
"arn:aws:iam::ACCOUNT_ID:role/ALB_CONTROLLER_ROLE_NAME"
  region: "us-east-1"
  vpcId: "vpc-xxxxxxxxxxxxxxxxx"
```

This controller is the foundation for the other ingress options in the repository. NGINX, Traefik and Kong ask for an NLB with the same three annotations:

```
# ingress-nginx/values.yaml (excerpt)
service:
  type: LoadBalancer
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
```

With `nlb-target-type: ip`, the NLB sends traffic straight to the pod IP, without going through a node port. This works because on EKS the **VPC CNI** gives every pod a real VPC IP. Without the controller installed, these annotations do nothing, and the Service waits for a load balancer that never arrives.

Ingress and Gateway API

The **Ingress** is Kubernetes' older API for HTTP routing: host and path lead to a Service. It is simple, but small. Anything beyond that (timeouts, rewrites, authentication) becomes a controller-specific **annotation**, and switching controllers means rewriting the annotations.

The **Gateway API** is its replacement. It splits the work into three resources with different owners:

Resource	Owner	What it defines
`GatewayClass`	whoever installs the controller	which implementation (Envoy, Istio, Kong…)
`Gateway`	the platform team	port, protocol, certificate
`HTTPRoute`	the application team	host, path, weights, headers

Weighted traffic splitting, headers and mirroring are part of the specification, so they work the same on any controller.

The five ingress controllers

The repository ships five options. The full comparison is in `ingress-comparison.md`. In short:

Controller	Folder	Configuration	Best for
AWS ALB	`ingress-alb/`	Ingress + annotations	WAF, Cognito and ACM integration, no pod in the path
ingress-nginx	`ingress-nginx/`	Ingress + annotations	Legacy (see warning below)
Envoy Gateway	`ingress-envoy/`	Gateway API	New projects, open standard
Traefik	`traefik/`	Ingress + IngressRoute	Dynamic environments, built-in dashboard
Kong	`kong/`	Ingress + plugins	API gateway: per-consumer auth and rate limiting

ingress-nginx has been retired

In November 2025, the Kubernetes project announced the retirement of **ingress-nginx**. Maintenance ended in March 2026, and since then there are no new releases and no security fixes. Existing installations keep working.

The chart stays in the repository for study and for clusters that still depend on it. For a new cluster, prefer Envoy Gateway (Gateway API) or the ALB.

```
# ingress-nginx/values.yaml (excerpt)
controller:
  replicaCount: 2
  metrics:
    enabled: true
  resources:
    requests: { cpu: 100m, memory: 128Mi }
    limits:   { cpu: 500m, memory: 512Mi }
```

Envoy Gateway

```
# ingress-envoy/values.yaml
gateway-helm: {}
```

The file is empty on purpose. Envoy Gateway's defaults are already fit for production, and the real work lives in the `GatewayClass`, `Gateway` and `HTTPRoute` resources. An empty `values.yaml` is also a message: "we changed nothing from the default".

Traefik

```
# traefik/values.yaml (excerpt)
service:
  spec:
    type: LoadBalancer
providers:
  kubernetesIngress: { enabled: true }
  kubernetesCRD:     { enabled: true }
api:
  dashboard: true
ingressRoute:
  dashboard:
    enabled: false
```

Traefik reads both formats: the regular Ingress and its own CRD, `IngressRoute`, which has middlewares (authentication, headers, rate limiting). The dashboard is on (`api.dashboard`) but **not published** (`ingressRoute.dashboard.enabled: false`): reach it with `kubectl port-forward`. Note `service.spec.type` too: up to chart 32 it was `service.type`. In the upgrade to chart 42 the old name became silently ignored, which is why a misnamed value is the number one risk of an upgrade (module 1).

Kong

```
# kong/values.yaml (excerpt)
ingressController:
  enabled: true
admin:
  enabled: false
serviceMonitor:
  enabled: true
```

- `admin.enabled: false` turns off Kong's admin API. With the ingress controller on, configuration comes from Kubernetes (CRDs such as `KongPlugin`), and an open admin API would be one more door to attack.
- `serviceMonitor.enabled: true` creates a ServiceMonitor for the Prometheus in module 4. That type only exists after `prometheus-stack`, so install Prometheus before Kong.

ExternalDNS: automatic records

ExternalDNS reads the cluster's Ingresses and Services and creates the matching DNS records in Route 53. You publish `app.example.com` in an Ingress, and the record appears pointing at the load balancer, without opening the console.

```
# external-dns/values.yaml
external-dns:
  provider:
    name: aws
  domainFilters:
    - "example.com"
  policy: sync
  txtOwnerId: "CLUSTER_NAME"
```

- `domainFilters` limits the zones it may touch. Without a filter, it acts on every zone the role can see.
- `policy: sync` creates **and deletes** records. Delete the Ingress and the record goes away. Use `upsert-only` if it must never delete, for example during a migration.
- `txtOwnerId` tags each record with an owner TXT record. ExternalDNS only deletes records carrying its own tag. That protects hand-made records and lets two clusters share a zone without deleting each other's records, **as long as each cluster uses a different ID**.

cert-manager: automatic certificates

cert-manager issues and renews TLS certificates inside the cluster. You describe what you want in a `Certificate` resource (or an Ingress annotation), and it talks to the certificate authority, stores the certificate in a Secret and renews it before it expires.

Who issues is defined in a **ClusterIssuer**, applied after the chart:

```
# cert-manager/manifests/cluster-issuer.yaml (excerpt)
kind: ClusterIssuer
metadata:
  name: letsencrypt-production
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: "admin@example.com"
    solvers:
      - dns01:
          route53:
            hostedZoneID: "Z0XXXXXXXXXXXXX"
            region: "us-east-1"
```

How it works

(1) Request

An Ingress with the `cert-manager.io/cluster-issuer: letsencrypt-production` annotation asks for a certificate for its host.

(2) DNS-01 challenge

Let's Encrypt asks for proof that you control the domain. With **DNS-01**, cert-manager creates an `_acme-challenge` TXT record in Route 53 (hence the IRSA role), Let's Encrypt checks it, and the record is

removed.

(3) Issuance and renewal

The certificate goes into a Secret that the ingress controller uses. Let's Encrypt certificates last 90 days, and cert-manager renews about 30 days before expiry.

DNS-01 has two advantages over **HTTP-01**: it works for services that are not exposed to the Internet, and it is the only one that issues **wildcard** certificates (`*.example.com`).

The ClusterIssuer name must match everywhere it is used. This repository's Uptime Kuma pointed at `letsencrypt-prod`, an issuer that does not exist, and was fixed to `letsencrypt-production`. With the wrong name, the certificate stays pending forever, with no error from Helm.

The Prometheus in module 4 has the `CertificateExpiringSoon` alert, which fires when a certificate has less than 7 days left. If it fires, automatic renewal failed days earlier.

Summary

Karpenter creates right-sized nodes for pending pods and removes what is left over. The chart installs the controller, and the **NodePool** and **EC2NodeClass** say what it may create: Spot or on-demand, `c`, `m` and `r` families, a 100-vCPU ceiling and consolidation after 30 seconds. Subnets need the discovery tag, and the controller must not run on nodes it manages.

Disks come from **StorageClasses**: encrypted **gp3**, created in the pod's zone, for almost everything, and **EFS** when several pods need the same volume.

The **AWS Load Balancer Controller** is the foundation of inbound traffic: it creates the ALB for Ingresses and the NLB for Services, and NGINX, Traefik and Kong depend on it. Among the controllers, prefer the **Gateway API** (Envoy Gateway) or the ALB for a new cluster, because **ingress-nginx** was retired in March 2026.

Finally, **ExternalDNS** publishes records in Route 53 and only deletes what it marked as its own, and **cert-manager** issues and renews certificates through the **DNS-01** challenge, with a ClusterIssuer whose name must match everywhere.

Quiz

You applied the NodePool and the EC2NodeClass, there are Pending pods, and no node is created. Karpenter's log says it found no subnets. What is missing?

- Tagging the private subnets (and the security groups) with `karpenter.sh/discovery: <cluster>`. That is how the EC2NodeClass finds where to launch nodes.

A broken deploy asked for 2,000 replicas. What stops Karpenter from creating hundreds of nodes?

- The NodePool's `limits.cpu: "100"`. The vCPUs of that NodePool's nodes add up to 100 at most.

A pod with a gp3 volume went Pending with a zone error after being rescheduled. Which StorageClass setting keeps the disk from being created in the wrong zone in the first place?

- `volumeBindingMode: WaitForFirstConsumer`. The disk is created only after the pod has a node, in that node's zone.

Three pods in different zones need to write to the same directory. gp3 or EFS?

- EFS, with `ReadWriteMany`. An EBS volume lives in one zone and accepts one node at a time.

You installed Traefik and its `LoadBalancer` Service has had no address for ten minutes. Events show no error. What is probably missing?

- The AWS Load Balancer Controller. The `aws-load-balancer-type: external` annotations only take effect with it installed.

Which ingress controller do you recommend for a new cluster in 2026 that needs weighted traffic splitting, and why not ingress-nginx?

- Envoy Gateway, with the Gateway API (`HTTPRoute` with weights). ingress-nginx was retired in March 2026 and no longer gets security fixes.

Two clusters publish records to the same Route 53 zone with ExternalDNS, and one started deleting the other's records. What is wrong?

- Both use the same `txtOwnerId`. Each cluster needs its own ID so it only deletes what it created.

You need a `*.example.com` certificate for an internal service with no Internet access. Which ACME challenge do you use?

- DNS-01. HTTP-01 requires Let's Encrypt to reach the service over the Internet and does not issue wildcard certificates.

The `CertificateExpiringSoon` alert fired. What does that say about renewal?

- That it failed: cert-manager renews about 30 days before expiry, and the alert only fires with less than 7 days left. Check the `Certificate` resource and the challenge events.

Additional resources

- Karpenter: NodePools
- Karpenter: Disruption
- Amazon EBS CSI driver
- Amazon EFS CSI driver
- AWS Load Balancer Controller
- Gateway API
- Ingress NGINX retirement (Kubernetes blog)
- ExternalDNS on AWS
- cert-manager: ACME DNS-01 with Route 53